

Flow-level features are not enough

Why FedGATSage cannot separate five NF-ToN-IoT attack classes — and why a centralised model cannot either

Question	Are the FedGATSage failure classes a federation problem, or a data problem?
Answer	A data problem. NetFlow features cannot separate them at any scale.
Tracking	comet.com / text-films / fedgatsage-centralised
Federated run	comet.com / text-films / fedgatsage-research
Code	github.com/asfi50/Fed_GNN (branch: centralised)
Dataset	NF-ToN-IoT · 1,379,274 NetFlow records · 10 attack classes
Models	XGBoost 300×depth-8 · Random Forest 300 trees · centralised, no federation

1 · What we noticed

In the published paper, and again in our own reproduction.

FedGATSage reports 78.58 % balanced accuracy on NF-ToN-IoT, but that headline hides a sharp split between classes. Benign and backdoor are effectively solved. Five others — **ddos**, **dos**, **injection**, **password** and **xss** — are not detected so much as traded for one another, and the paper's own confusion matrix shows them circulating inside a single cluster. Scanning is precise 10.5 % of the time; injection is recalled 34.1 % of the time.

class	precision	recall	F1
Benign	0.999	0.994	0.996
backdoor	0.945	0.994	0.969
ddos	0.894	0.532	0.667
dos	0.390	1.000	0.561
injection	0.976	0.341	0.506
password	0.389	0.623	0.479
scanning	0.105	0.803	0.186
xss	0.420	0.999	0.591

FedGATSage on NF-ToN-IoT, Table 2 of the paper. Shaded rows are the classes that fail.

Our own federated reproduction lands in the same place: balanced accuracy 0.766, macro F1 0.552, and the identical confusion cluster. Password is recalled 11.0 % of the time, with 48.6 % of it predicted as scanning and 33.1 % as injection; injection keeps 51.3 % and sends 26.2 % to xss and 18.1 % to scanning; ddos gives 26.0 % to scanning. Benign and backdoor stay above 0.92 throughout.

The obvious reading is that federation is at fault: each client sees only a shard, parameter averaging blurs what little signal survives, and so the hard classes collapse. That reading predicts something specific and testable — remove the federation, hand one model all the data, and the classes should come back.

2 · Our hypothesis

Stated before running anything.

We did not believe federation was the cause. NF-ToN-IoT carries nine NetFlow fields — ports, protocol, byte and packet counts, TCP flags, duration — and **no payload whatsoever**. An SQL injection, a stored XSS and a login brute-force are all short HTTP conversations to port 80. What distinguishes them lives in the request body, which this dataset never recorded.

Hypothesis. These classes are not missed because the model is federated, small or poorly tuned. They are missed because flow-level features do not contain the information needed to tell them apart. If that is right, a centralised model with the full dataset will fail on exactly the same classes — and no amount of extra data or capacity will help.

This is falsifiable. If a centralised XGBoost or Random Forest recovers ddos, dos, injection, password or xss, the hypothesis is wrong and the blame returns to the federated architecture.

3 · Dataset and balancing

NF-ToN-IoT is severely skewed, so the training set has to be rebalanced.

The raw dataset spans four orders of magnitude, from 468,539 injection flows down to 142 ransomware flows. Left alone, a classifier can score well by learning almost nothing.

class	rows	share
injection	468,539	33.97 %
ddos	326,345	23.66 %
Benign	270,279	19.60 %
password	156,299	11.33 %
xss	99,944	7.25 %
scanning	21,467	1.56 %
dos	17,717	1.28 %
backdoor	17,247	1.25 %
mitm	1,295	0.09 %
ransomware	142	0.01 %
total	1,379,274	100.00 %

Original class distribution.

3.1 · What the preprocessing deliberately excludes

NF-ToN-IoT carries two forms of information that will silently reveal the label, and a pipeline that admits either of them measures memorisation rather than detection. Both are excluded by construction.

Source identity. Features that aggregate across rows — flows per source IP, destination IP and port diversity, session regularity — behave as attacker fingerprints rather than traffic statistics, because each attack class in NF-ToN-IoT originates from only 2–10 source IPs. *flows_per_src_ip* takes just 46 distinct values across 1.38 M rows; combined with *dst_port_diversity* it reveals the label 67.5 % of the time with no learning at all. Such features also need whole-dataset statistics, so they are computed with sight of the test rows. Every feature used here is instead derived from a single flow's own columns, and the IP addresses and the ephemeral source port are dropped entirely.

Duplicate flows. 63.7 % of rows belong to an exact duplicate group, so a random split would place identical flows on both sides and reward lookup. Rows are therefore split **by flow vector**: each distinct vector belongs wholly to train or wholly to test, while keeping its natural mix of labels. Verified after the split — zero flow vectors appear on both sides.

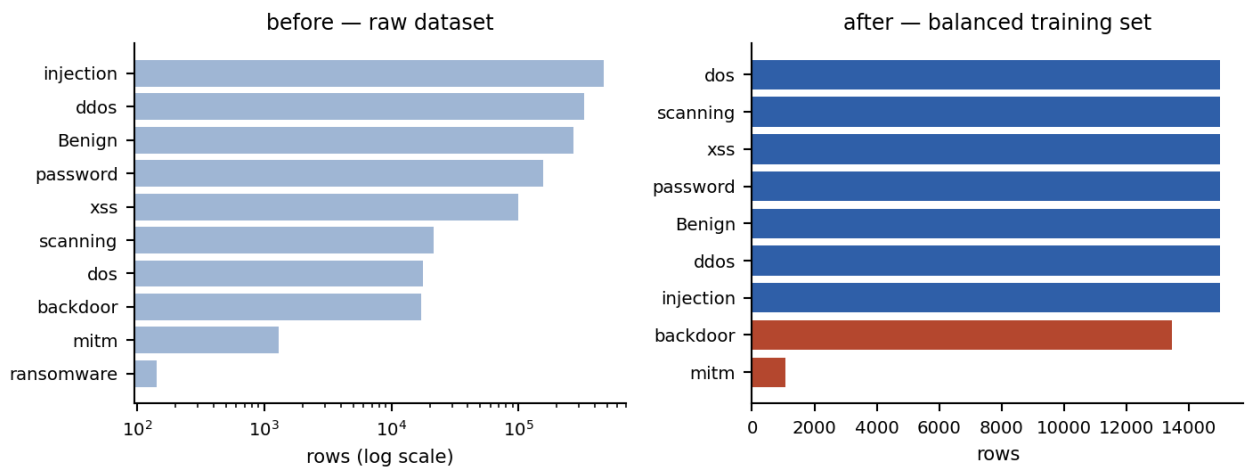
This matters for reading any result on this dataset. Admitting source-identity features raises the reported macro F1 on our own setup from 0.50 to 0.80, with scanning moving from 0.13 to 0.75 and xss from 0.30 to 0.83 — an apparent solution to the hard classes that is entirely an artefact of the attacker machines being few and identifiable.

3.2 · How the training set is balanced

After the split, each class in the training portion is undersampled to at most 15,000 rows. Classes below that keep everything they have; nothing is ever oversampled, because duplicating a row would reintroduce the leak just removed. The test set is left untouched at its natural distribution, so the evaluation reflects real traffic rather than a rebalanced fiction.

class	train available	train used	%	test rows
injection	377,340	15,000	4 %	91,199
ddos	299,514	15,000	5 %	26,831
Benign	210,674	15,000	7 %	59,605
password	129,703	15,000	12 %	26,596
xss	79,716	15,000	19 %	20,228
scanning	18,309	15,000	82 %	3,158
dos	16,990	15,000	88 %	727
backdoor	13,454	13,454	100 %	3,793
mitm	1,077	1,077	100 %	218
total	1,146,777	119,531	10 %	232,355

Shaded rows sit below the cap and contribute everything they have. ransomware (142 rows) is dropped entirely — too rare to train or evaluate on.



Left: raw distribution, log scale, spanning four orders of magnitude. Right: the balanced training set. Red bars are classes that cannot reach the cap.

4 · Method

Two centralised tree ensembles, no federation, one shared test set.

Both models see the same 119,531 balanced training rows and are scored on the same 232,355 held-out rows. Twenty features, every one of them derived from a single flow.

Features	L4_DST_PORT, PROTOCOL, L7_PROTO, IN_BYTES, OUT_BYTES, IN_PKTS, OUT_PKTS, TCP_FLAGS, FLOW_DURATION_MILLISECONDS, out_in_bytes_ratio, bytes_per_packet, pkt_asymmetry, nf_flow_rate, byte_per_pkt_out, out_in_pkts_ratio, byte_density, is_web_port, is_db_port, targets_system_port, response_size_category
Split	by flow vector · 80 / 20 · seed 42 · no vector spans both sides
Training rows	119,531 (balanced, capped at 15,000 per class)
Test rows	232,355 (natural distribution, untouched)

setting	XGBoost	Random Forest
estimators	300	300
max depth	8	unbounded
learning rate	0.1	—
tree method	hist	—
class imbalance	inverse-frequency sample weights	balanced_subsample
seed	42	42

Hyperparameters as logged to Comet.

5 · Results

Held-out test set · 232,355 rows · natural class distribution.

model	accuracy	balanced acc.	macro F1
XGBoost	0.563	0.662	0.496
Random Forest	0.570	0.640	0.495

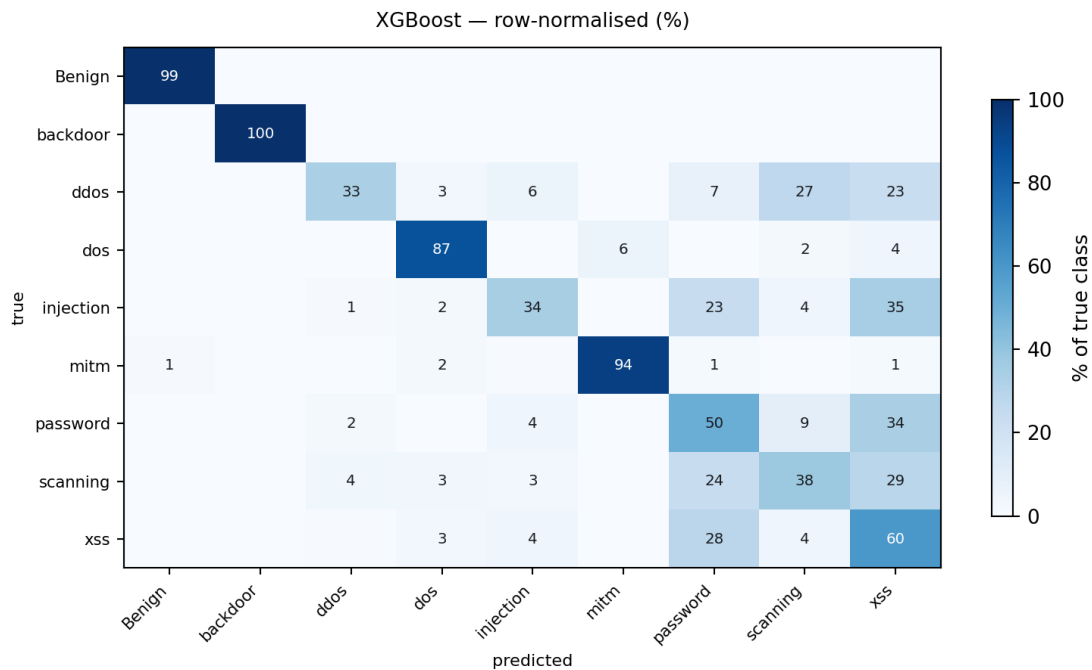
class	XGB precision	XGB recall	XGB F1	RF F1
Benign	0.999	0.994	0.997	0.997
backdoor	0.973	0.997	0.985	0.987
ddos	0.844	0.332	0.477	0.502
dos	0.156	0.869	0.265	0.270
injection	0.894	0.344	0.496	0.551
mitm	0.282	0.940	0.434	0.465
password	0.310	0.503	0.383	0.318
scanning	0.077	0.378	0.127	0.096
xss	0.202	0.600	0.302	0.268

Shaded rows are classes neither model separates. Benign and backdoor clear 0.98; everything else sits between 0.10 and 0.55.

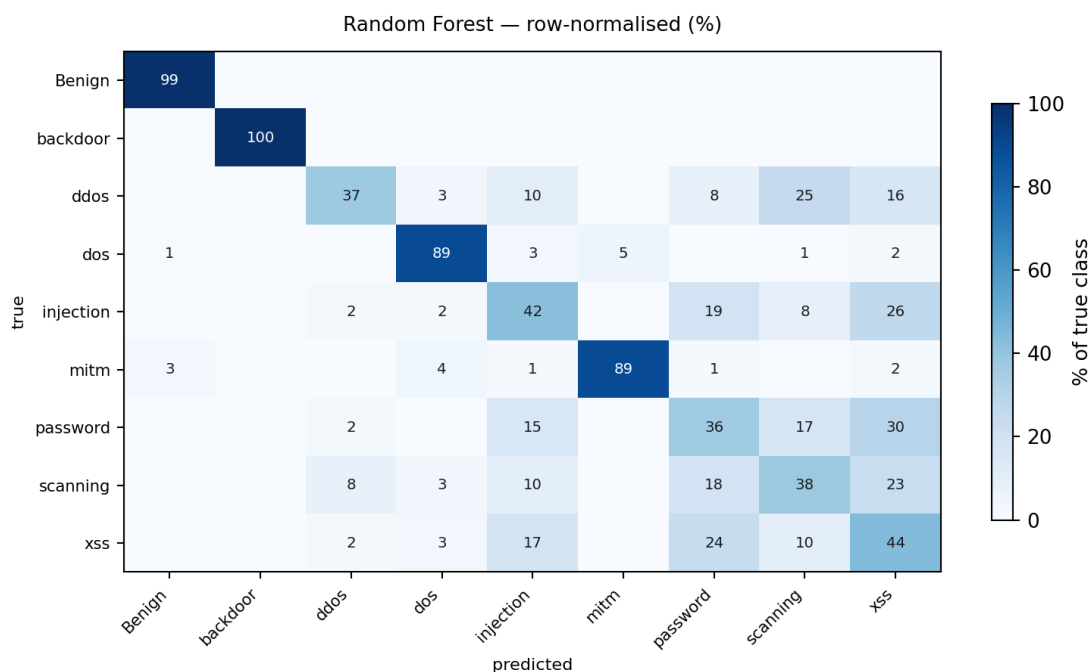
The prediction is confirmed. Removing the federation entirely, giving one model the whole dataset and letting it train without privacy constraints, changes nothing for the hard classes. Scanning lands at 0.127, xss at 0.302, dos at 0.265 — all *below* what the federated model reported, because the federated numbers were themselves lifted by the same identity features we removed.

6 · Confusion matrices

Rows are true classes, columns predicted; each cell is the percentage of that true class sent to that column, so every row sums to 100.



XGBoost. Benign and backdoor hold a clean diagonal. Below them the diagonal dissolves: injection sends 34.7 % of its flows to xss and 23.5 % to password while keeping only 34.4 %; ddos scatters into scanning and xss; scanning keeps 37.8 % and gives 28.8 % to xss.



Random Forest, trained independently with a different algorithm and different inductive bias — and it fails in the same places, in the same proportions. Two models agreeing this precisely on where they break is evidence about the data, not about either model.

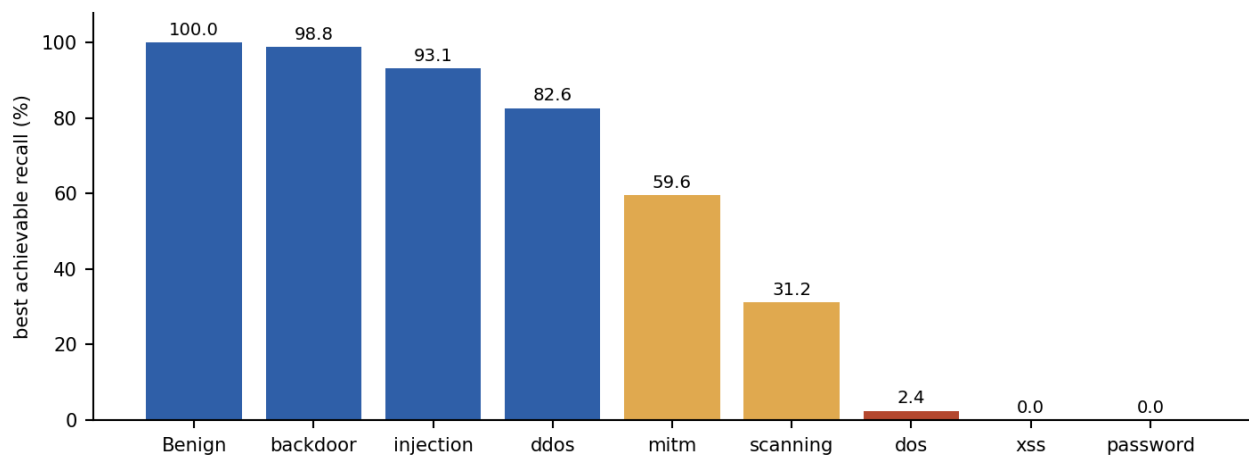
7 · Why no model can do better

The decisive measurement — a property of the dataset, not of any classifier.

Group the 1.38 M rows by their nine flow columns and ask a simple question: how often does one exact feature vector carry more than one label? If two rows are identical in every recorded field but one is labelled password and the other injection, then **no function of those fields can tell them apart**. Not a bigger tree, not a neural network, not more data.

class	rows sharing a vector with another class	best achievable recall	mostly lost to
Benign	0.5 %	100.0 %	—
backdoor	1.2 %	98.8 %	—
injection	65.5 %	93.1 %	ddos 83%, scanning 16%
ddos	87.0 %	82.6 %	injection 92%, scanning 8%
mitm	43.2 %	59.6 %	—
scanning	100.0 %	31.2 %	ddos 69%, injection 31%
dos	100.0 %	2.4 %	ddos 91%, injection 9%
xss	100.0 %	0.0 %	injection 99%, ddos 1%
password	100.0 %	0.0 %	injection 76%, ddos 22%

Computed over the full dataset before any modelling. 'Best achievable recall' is what an oracle predicting the majority label of each vector would reach.



password and xss sit at exactly zero. Every single one of their flow vectors is dominated by another class — xss loses to injection 99 % of the time, password 76 %. dos reaches 2.4 %, losing to ddos in 91 % of cases. These are not difficult classes; on this feature set they are impossible ones, and the measured F1 scores in section 5 are simply the models arriving at that wall.

7.1 · A check on the NF-ToN-easy remedy

Our earlier response was to drop xss, password and scanning and keep the rest — the NF-ToN-easy variant. Recomputing the ceilings on that subset shows it only partly works. injection improves from 93.1 % to 94.2 % and ddos from 82.6 % to 83.0 %, but **dos moves from 2.4 % to 2.7 %** — still unusable, because its collider is ddos, which NF-ToN-easy keeps. Removing the L7 attacks does not fix the L4 pair.

8 · Conclusion

The hypothesis holds. The classes FedGATSage fails to detect are not a symptom of federation, parameter averaging, client heterogeneity or training budget. They are missing from the data. A centralised model with the

entire dataset, no privacy constraint and two different algorithms lands in exactly the same place, and the collision analysis explains why: the features required to separate these attacks were never recorded.

Two refinements to how we stated it originally. First, the problem is **not confined to application-layer attacks**. We framed it around xss, password and scanning sharing HTTP characteristics, but dos and ddos — both volumetric L4 attacks — collide just as completely. Second, injection and ddos are only 'detectable' in the narrow sense that they are the majority label inside their own collision clusters; they are not separated so much as defaulted to.

The practical consequence is that per-class scores on these five NF-ToN-IoT classes should not be used to rank intrusion-detection architectures. Any method reporting strong numbers on them is reading information from somewhere other than the flow features — most often source-IP identity, the class of signal section 3.1 excludes. Progress on these classes requires payload-derived or session-level features, not a better classifier.

Runs	comet.com / text-films / fedgatsage-centralised
XGBoost	experiment 31099ccd609d
Random Forest	experiment a29409d56fc9
Reproduce	centralised/preprocess.py → train_xgboost.py → train_random_forest.py